

Polynomial Interpolation Report

Comparative Analysis: Lagrange, Newton, and Hermite

Youssef Benzemroun

GitHub repository: [Click here to access the source code](#)

1 Introduction

In this practical assignment, we study the polynomial interpolation of two functions:

- $f(x) = x^3 + 4$
- $g(x) = \ln(x + 1)$

The goal is to compare three interpolation methods: Lagrange polynomials, Newton's method, and Hermite interpolation. We analyze the evolution of the error as we increase the number of interpolation points, while observing the possible appearance of Runge's phenomenon.

2 Libraries and Tools Used

2.1 SciPy and the `scipy.interpolate` module

SciPy is a Python library dedicated to scientific computing. It builds on NumPy and provides many numerical algorithms. The `scipy.interpolate` submodule contains interpolation functions, including polynomial, spline, and Hermite interpolation.

- **KroghInterpolator**: a SciPy class that constructs an interpolation polynomial using Newton's method (divided differences). The abscissas must be strictly increasing.
- **CubicHermiteSpline**: a SciPy function that performs piecewise cubic Hermite interpolation. It uses the function values and its derivatives at the nodes, and therefore requires providing `dydx`.

2.2 Manual Implementation (Lagrange)

For the Lagrange method, we implemented the classical interpolation polynomial formula as a Python function, without relying on an external library.

3 Source Code

The Python code used to generate the plots and perform the calculations is shown below.

```
[ ] import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import KroghInterpolator, CubicHermiteSpline

[ ] def f(x):
    return x**3 + 4

def g(x):
    return np.log(x + 1)
#fcts de derives pour hermit
def df(x):
    return 3 * x**2

def dg(x):
    return 1 / (x + 1)

#fct de lagrange
def lagrange(x, x_points, y_points):
    x = np.asarray(x, dtype=float)
    rslt = np.zeros_like(x)
    for i in range(len(x_points)):
        L = np.ones_like(x)
        for j in range(len(x_points)):
            if i != j:
                L *= (x - x_points[j]) / (x_points[i] - x_points[j])
        rslt += y_points[i] * L
    return rslt

x_plot = np.linspace(0, 3, 500)
valeurs = [3, 5, 10, 15]

#fcts de newton et hermit
for func, dfunc, name in [(f, df, 'f(x) = x3 + 4'),
                          (g, dg, 'g(x) = ln(x+1)')]:
    plt.figure(figsize=(12, 8))
    plt.plot(x_plot, func(x_plot), 'k-', label='Fonction originale', linewidth=2)

    for N in valeurs:
        x_points = np.linspace(0, 3, N)
        y_points = func(x_points)

        y_lagrange = lagrange(x_plot, x_points, y_points)
        plt.plot(x_plot, y_lagrange, '--', label=f'Lagrange N={N}')
```

Figure 1: First part of the Python source code.

```
newton_interp = KroghInterpolator(x_points, y_points)
plt.plot(x_plot, newton_interp(x_plot), ':', label=f'Newton N={N}')

dy_points = dfunc(x_points)
hermite_interp = CubicHermiteSpline(x_points, y_points, dy_points)
plt.plot(x_plot, hermite_interp(x_plot), '-.', label=f'Hermite N={N}')

plt.title(f'Interpolation de {name}')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```

Figure 2: Second part of the Python source code.

4 Runge's Phenomenon

Before commenting on the curves, it is essential to define Runge's phenomenon. It is a mathematical problem that occurs when using global polynomial interpolation (such as Lagrange or Newton) with a large number of equidistant points.

Contrary to intuition, increasing the number of points does not always improve the interpolation. The polynomial begins to oscillate more and more strongly at the ends of the interval, even if the original function is very smooth. The maximum error explodes near the boundaries. In summary, Runge's phenomenon is characterized by undesirable oscillations at the edges as the degree of the polynomial increases.

5 Analysis of Results

5.1 Case of $f(x) = x^3 + 4$

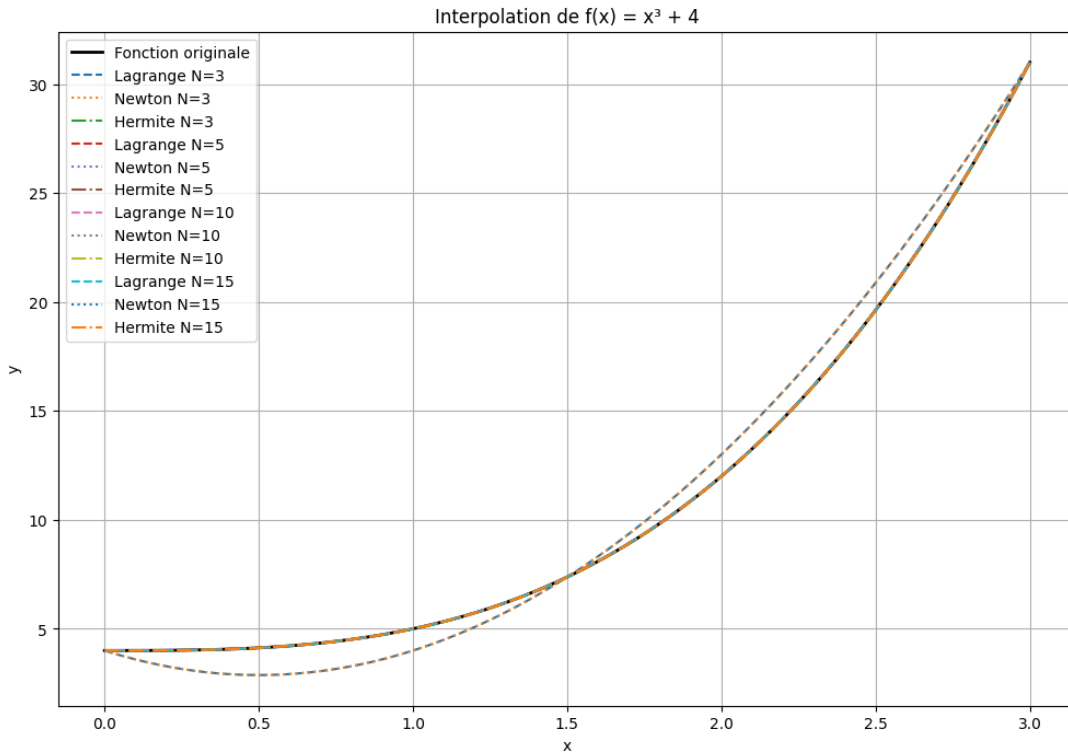


Figure 3: Interpolation of $f(x) = x^3 + 4$.

Observing Figure 3, the curves almost perfectly overlap. This is because $f(x)$ is already a polynomial of degree 3. As soon as we use $N \geq 4$ points, the interpolation polynomial (Lagrange or Newton) is exactly identical to $f(x)$. For $N = 3$, a slight difference appears at the beginning (the curve goes a bit lower), but as N increases, everything falls into place. **There is therefore no Runge phenomenon here**, because the function is already a polynomial.

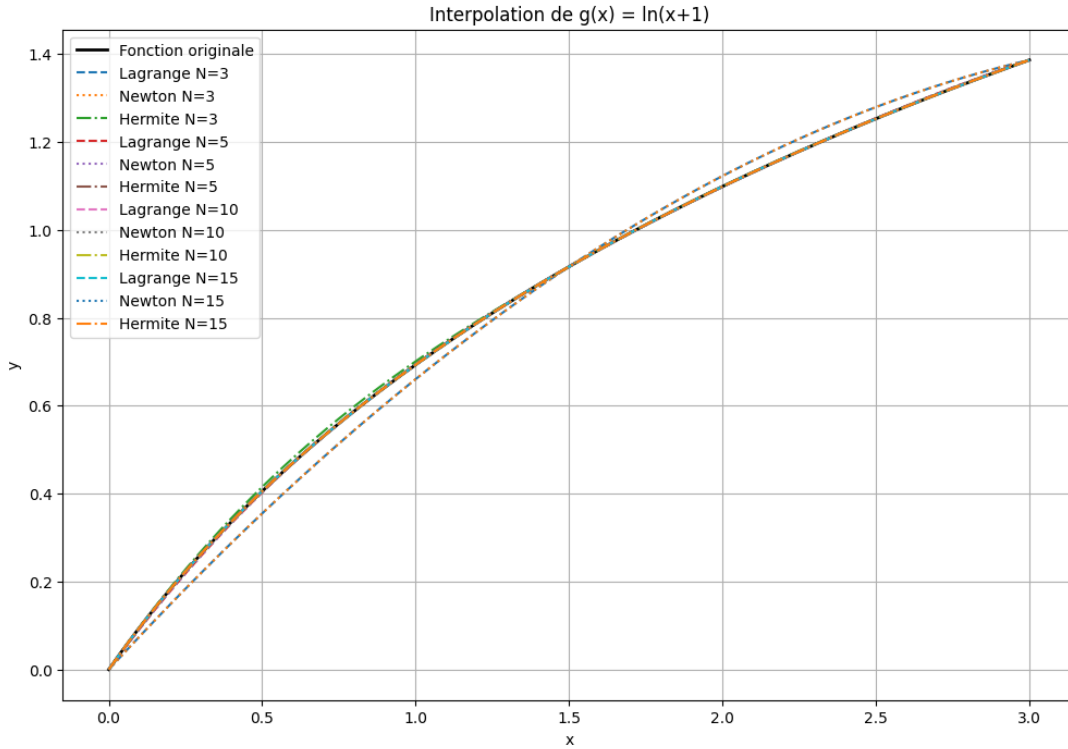


Figure 4: Interpolation of $g(x) = \ln(x + 1)$.

5.2 Case of $g(x) = \ln(x + 1)$

In Figure 4, things are different because $g(x)$ is not a polynomial. With $N = 3$ and $N = 5$, the interpolation is quite coarse. However, looking carefully at the curves for $N = 10$ and $N = 15$ (especially near $x = 0$ and $x = 3$), we begin to see very slight oscillations or small deviations from the black curve.

On the interval $[0, 3]$, Runge's phenomenon remains **very moderate**. This is because $\ln(x+1)$ has derivatives that do not grow too quickly on this interval, and because $N = 15$ is not yet a high enough degree to trigger catastrophic oscillations. If we had taken a larger interval (for example $[-0.9, 10]$) or $N = 30$, we would have seen the Lagrange and Newton curves start to "blow up" completely at the ends.

6 Comparison of Methods

- **Lagrange vs Newton:** In our plots, they give exactly the same results (the curves are superimposed). This is logical, since they construct the same unique polynomial, but with different algorithms. Newton is simply more practical to compute and implement programmatically.
- **Hermite (CubicHermiteSpline):** This is the best-performing method here. Even for $g(x)$ with $N = 3$, the Hermite curve (dashed) is very close to the function. Because it is a piecewise interpolation (cubic splines), it does not suffer from Runge's phenomenon. It remains smooth and local, without oscillations at the edges, even if we increased N or the interval size.

7 Conclusion

Increasing the number of points improves the interpolation for $f(x)$ (a polynomial), but for $g(x)$ (a transcendental function), it reveals the limits of global polynomial interpolation. Runge's phenomenon is a true mathematical limitation. To avoid it, it is better to use Chebyshev points (instead of equidistant points) or switch to splines such as Hermite interpolation, which offers much better stability.

Link to GitHub Repository

The complete source code, images, and report are publicly available at the following address:

https://github.com/Usiief/rapport-interpolation-H_L_N-